Formula 1 Podium Prediction Companion

Milestone 1 — analytical report

1. Summary

We predict, from information available **before the race starts**, which drivers finish on the podium. The work runs in one notebook (EDA → cleaning → data-engineering questions → feature engineering → preprocessing → modelling → experiments → XAI → inference). Splits are chronological (train 2003–2018, validation 2019–2021, test 2022–2024); the primary metric is average precision (AP, reported as PR-AUC) because only about 14% of driver-races are podiums, and F1 at a validation-tuned threshold is the secondary metric.

model	split	pr_auc	roc_auc	f1	precision	recall
selected: random_forest	validation	0.746	0.941	0.734	0.724	0.744
grid-position baseline	validation	0.676	0.917	0.717	0.717	0.717
selected: random_forest	test	0.695	0.936	0.655	0.679	0.632
grid-position baseline	test	0.647	0.914	0.649	0.650	0.647

Table 1. Headline results. The model is selected on validation PR-AUC only; the test split is used for evaluation and diagnostic analysis, never for selecting preprocessing, models, hyperparameters or thresholds.

Main findings.

- The selected model improves held-out PR-AUC, ROC-AUC and F1, but not recall or top-3 hit rate (test PR-AUC 0.695 vs 0.647): most of the signal is already in the starting grid.
- Grid and qualifying carry the model. Removing both together costs 0.118 test PR-AUC, while removing either alone costs almost nothing because they are 0.97 correlated.
- Permutation importance, SHAP and LIME agree on the top features; the ranking is stable across validation/test and across model families. Feature importance describes what the **model** uses and is **not** evidence of causality.
- The test block (2022–2024) is a new technical era; scores are lower than on validation and the era effect is quantified in Section 7.

2. Data and cleaning

Source: 14 Ergast-style tables (races, results, qualifying, drivers, constructors, standings, …) covering 1950–2024. All cleaning decisions are logged in the notebook and exported as CSV.

metric	before	after
literal sentinel cells	160076	0
results rows	26759	26668
duplicated driver-race pairs	85	0

Table 2. Effect of cleaning. The raw files were hashed before and after and did not change.

Cleaning steps. (1) The literal `\N` sentinel (160,076 cells) was converted to true missing values; (2) numeric/date types were repaired; (3) all primary keys were validated and 23 foreign-key relationships were checked with 0 unmatched rows; (4) 85 duplicated driver-race pairs (shared drives in the early decades) were resolved by keeping the best classified row, then more laps, then the smaller result id, leaving exactly one row per (race, driver).

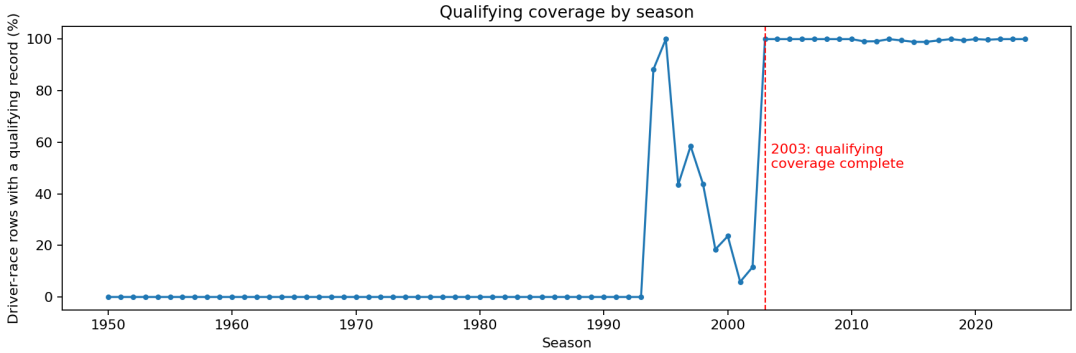


Figure 1. Qualifying coverage by season. No qualifying data exists before 1994 and only part of the field is recorded until 2002 — which is why the modelling table starts in 2003 (known limitation: qualifying features cannot be used for older seasons).

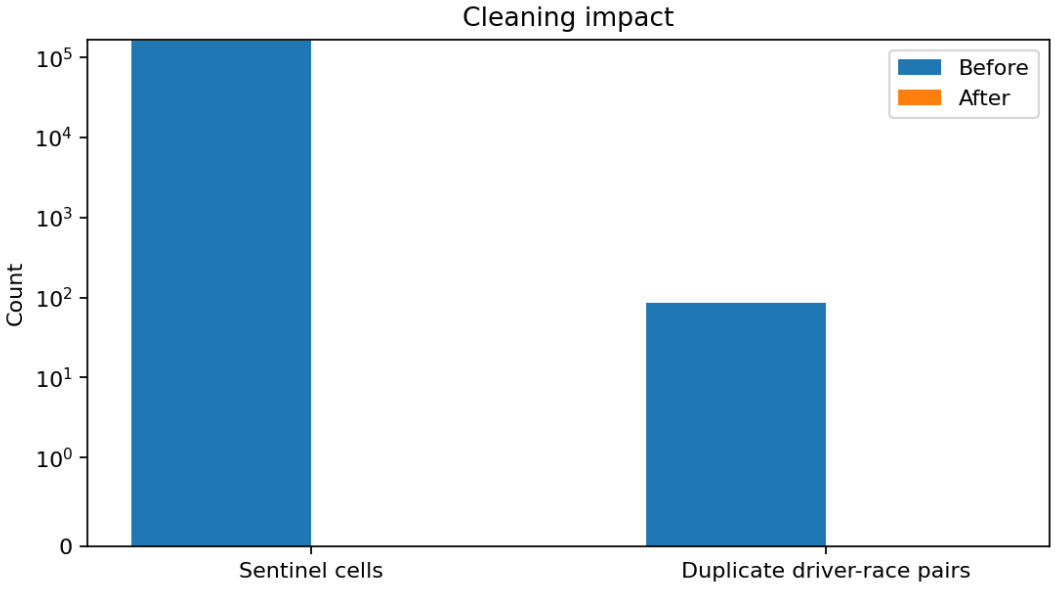


Figure 2. Impact of cleaning.

3. Data-engineering questions

Q1 — Which circuits convert a front-row start into a podium most often, and does the ranking change after 2014?

- Starting from the front row became more valuable overall: 68.0% of front-row starters finished on the podium in 2003–2013, against 76.0% in 2014–2024. This fits the long spells of one dominant team in the hybrid era.
- **Pre-hybrid (2003–2013) top converters:** Magny-Cours (11/12, 91.7%), Istanbul Park (78.6%), Monza and Monaco (77.3%).
- **Hybrid (2014–2024) top converters:** Suzuka (16/18, 88.9%), Yas Marina (86.4%), Monaco (85.0%), Barcelona and Spa (81.8%).
- **The ranking does change.** For the 14 circuits with at least 5 races in both eras, the rank correlation between eras is only 0.30 (p = 0.29). Monza fell from 1st to 10th of 14, Yas Marina jumped from 10th to 2nd, and Hungary sits last in the hybrid era (59%). Over the longer 1950–2013 vs 2014–2024 comparison the correlation is about zero (−0.03). Circuit "front-row reliability" is therefore not a stable property, and it should not be used as a fixed feature.
- **Qualifying ≠ grid.** 38 drivers qualified on the front row but started further back (penalties, pit-lane starts) and only 28.9% of them reached the podium. 36 drivers were promoted onto the front row and 63.9% of them did. At individual circuits the two definitions can differ a lot, for example Interlagos 2014–2024 is 78.9% by grid but 65.0% by qualifying. The grid is the more honest definition of a "front-row start".

Missingness: qualifying position is missing for only a handful of rows from 2003, so both definitions use the same races. Circuits with fewer than 5 races in an era are not ranked. **Cutoff:** both grid and qualifying position are known before the race, so the conditioning variable respects the cutoff. Only the outcome (podium) is post-race, which is what we are measuring.

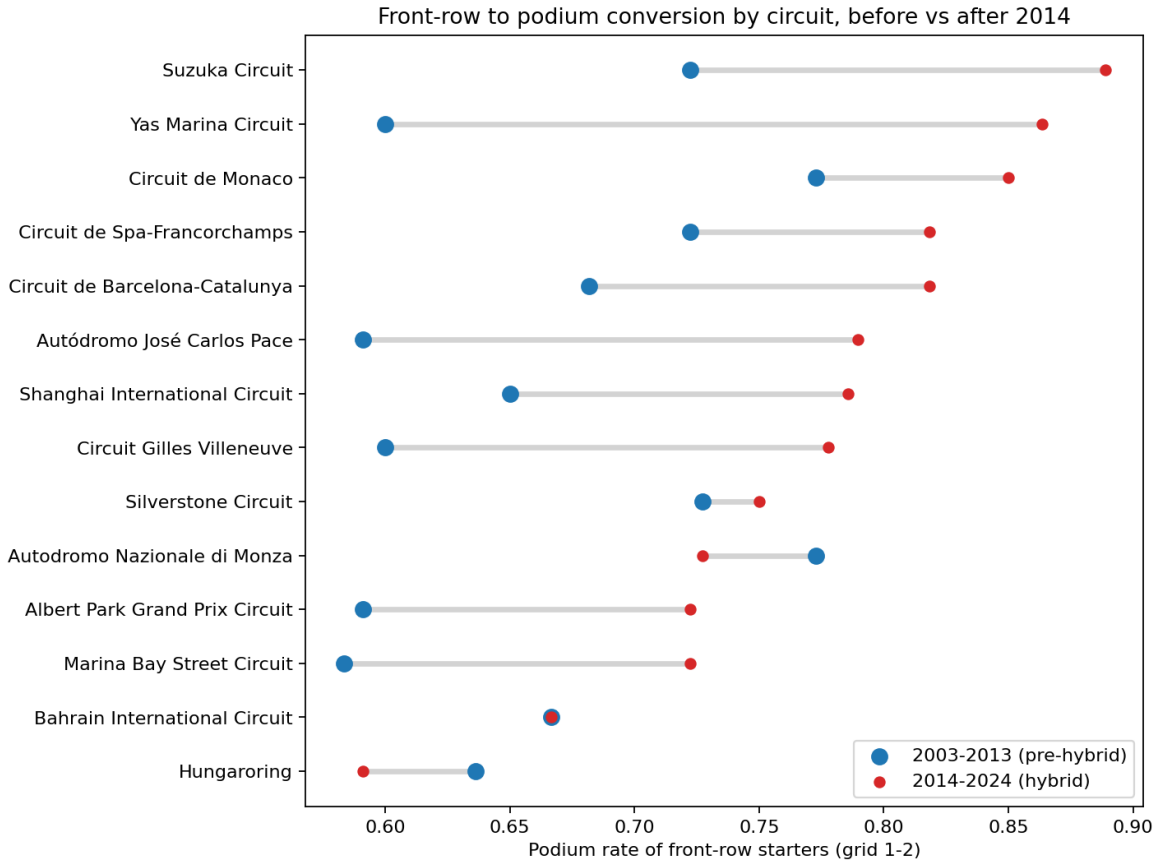


Figure 3. Front-row conversion by circuit, 2003–2013 vs 2014–2024 (Wilson intervals in the CSV).

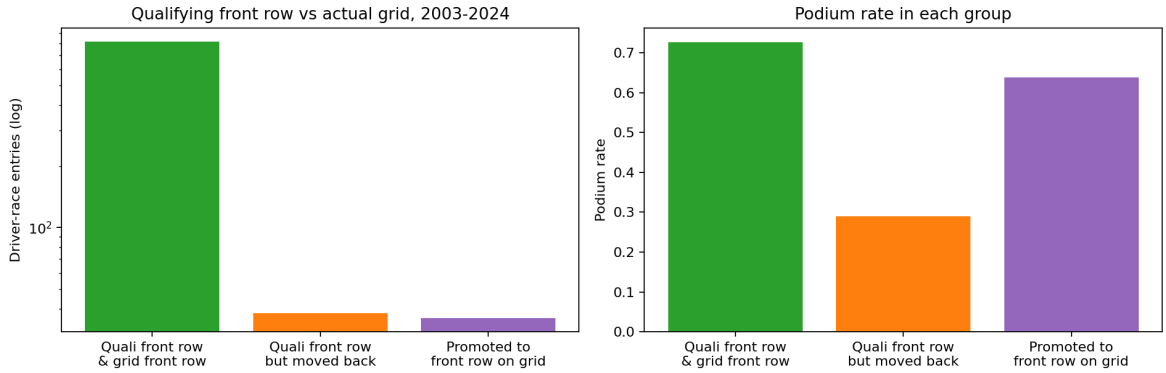


Figure 4. Front row by qualifying vs by grid: the definitions differ exactly in the penalty cases.

Q2 — Do drivers podium more often at circuits in their home country?

No, we find no evidence of a home podium advantage once starting position is controlled for.

- Without any adjustment, home drivers actually podium *less* often (11.8% vs 13.8%). They also start further back (median grid 13 vs 11), because home races attract one-off local entries and weaker local drivers. Comparing raw rates would be misleading.
- Within each grid band the home–away differences are small and change sign (home is better from P3 and P4–5, worse from P1 and P6–8).
- The grid-adjusted Mantel–Haenszel odds ratio is about 0.97, with a bootstrap 95% interval that comfortably contains 1. Home drivers scored 230 podiums against roughly 235 expected from their grid bands.
- The conclusion holds in every period and whether or not the Indianapolis 500 is included. Every interval contains 1.

Grain: driver-race rows. **Missingness:** 1,635 rows with grid 0 are left out because they cannot be placed in a grid band, and nationality and circuit country are complete. **Cutoff:** the home flag and grid are both known before the race. This result is also why `home_race` is *not* kept as a model feature (see the Feature Engineering part).

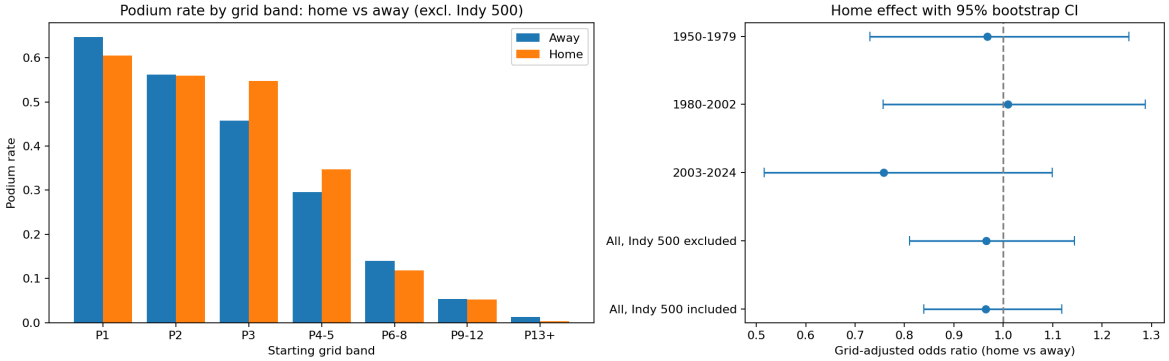


Figure 5. Raw home vs away podium rate and the grid-adjusted comparison.

Q3 — Which constructors have the highest mechanical-retirement rate in 2014–2021 vs 2022–2024?

- **2014–2021:** the highest mechanical-retirement rates belong to **Lotus F1 (17.3% of starts)**, **Toro Rosso (16.1%)**, **Renault (13.1%)** and **McLaren (13.0%)**. The McLaren figure comes mostly from the McLaren-Honda years. Mercedes (3.8%), Ferrari (4.7%) and the Alfa Romeo / Aston Martin entries are at the bottom.

- **2022–2024: Alfa Romeo (12.5%)** and **Alpine (10.4%)** are clearly the least reliable. Behind them the field is tightly packed between 2% and 7%.
- Reliability improved a lot overall. The whole grid went from about 12% mechanical DNFs per start in 2014 to about 3% in 2024. The top teams converged to roughly 4% in the new ground-effect era.
- **Sensitivity:** adding the ambiguous statuses barely changes the 2014–2021 ranking (rank correlation 0.97). For 2022–2024 it changes more (0.68), mainly because RB and Sauber only have one season (48 starts), so two or three ambiguous retirements move them up a lot. The top two teams stay the same either way.

Grain: race starts aggregated to constructor × period. **Missingness:** status is never missing, but `Retired` gives no reason (counted only in the broad version). **Caveat:** constructors are kept as they appear in the data, so renamed teams (Toro Rosso → AlphaTauri → RB, Force India → Racing Point → Aston Martin, Sauber → Alfa Romeo → Sauber, Renault → Alpine) show up as separate rows. **Cutoff:** this question is descriptive (retirements are a race outcome). It does not produce a feature for the same race, but the *past* DNF rate is used as a form feature below.

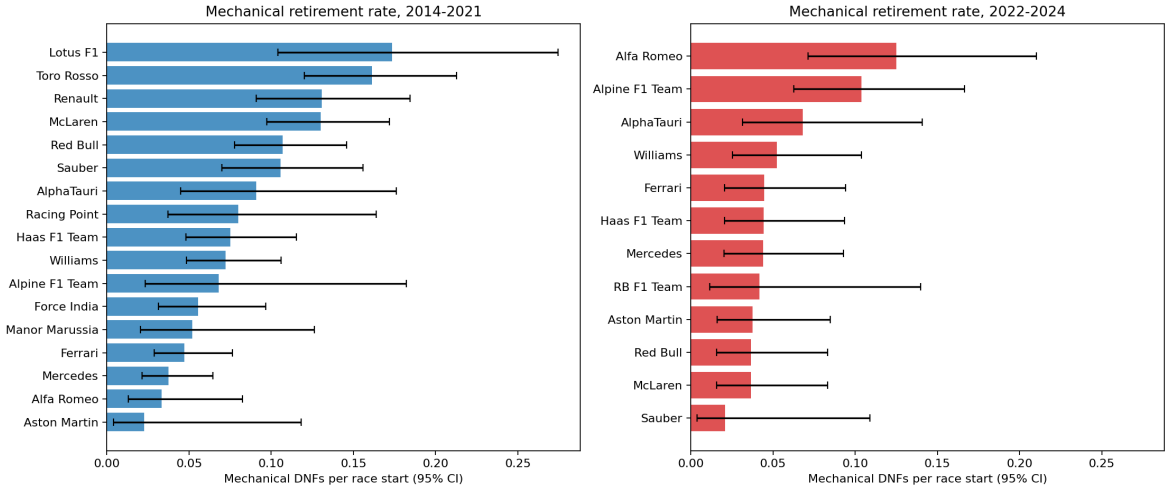


Figure 6. Mechanical-retirement rate by constructor and period.

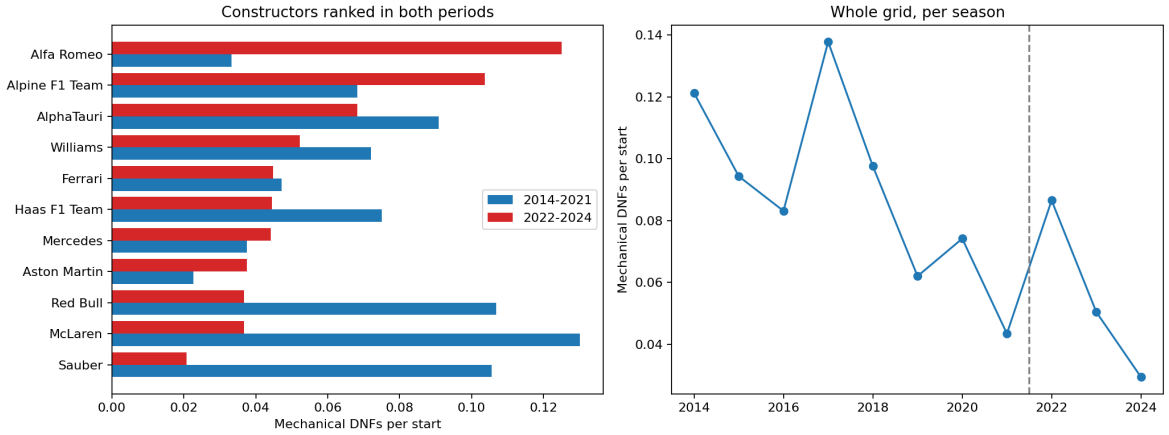


Figure 7. League-wide mechanical-DNF trend.

4. Feature engineering and selection

Every feature has to be computable for a race **before it starts**. We use three kinds of information:

- **Grid and qualifying:** known after qualifying, so allowed.
- **Form:** built only from races strictly *before* the current one (the `shift(1)` in every rolling feature).
- **Standings:** taken from the standings after the *previous round of the same season*, because the standings row for the current `raceId` already includes the current race.

Races are ordered by season and round to get a running race number (`race_seq`). Entries that **did not qualify** are dropped, because at the cutoff we already know they will not start, so they are not real prediction cases. Withdrawn entries (`W`) are kept: some withdrawals happen on race morning, after the cutoff, and dropping them would use information we would not have.

Window experiment and chosen features

Going from 3 to 10 races helps clearly (about +0.03 to +0.05 AUC). From 10 to 20 the average finish does not improve at all and the podium rate gains only ~0.006. A 20-race window covers almost a whole season, so after a team switch or a regulation change it would keep describing the old car for a year. We use **10 races** for all form features.

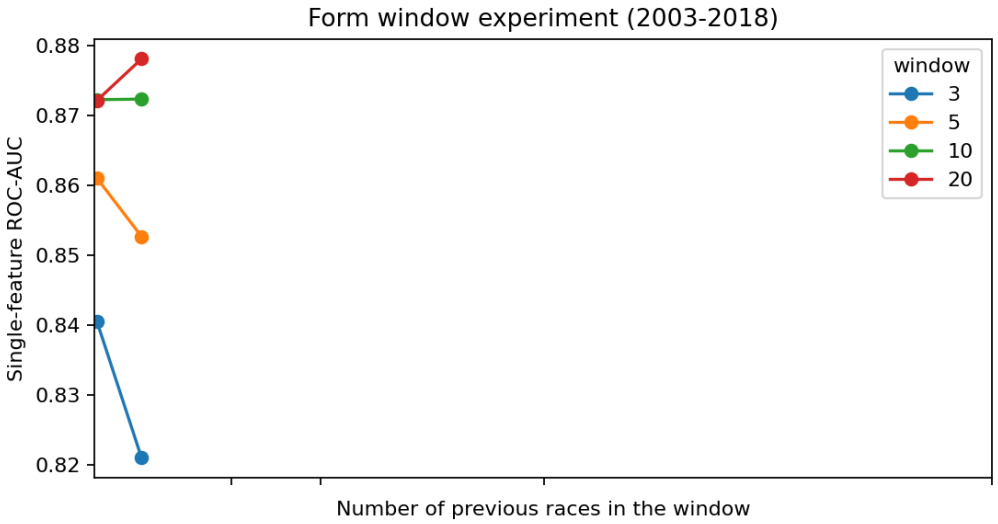


Figure 8. How many past races should the form window use? 10 was chosen (a 20-race window adds nothing and goes stale across team switches / regulation changes).

Final feature set and justification

Group	Feature	Why it is valid at the cutoff	Evidence
grid	`grid_pos`	Starting slot is fixed once penalties are applied, before the start	Strongest signal (AUC ≈ 0.91), steep non-linear podium curve
grid	`grid_penalty`	Grid and qualifying are both known before the start	Top-5 qualifiers moved back: 24% vs 50% podium rate (and Q1)
qualifying	`quali_pos`	Qualifying is over at the cutoff	AUC ≈ 0.91. Differs from grid exactly in the penalty cases
qualifying	`quali_gap_pct`	Qualifying lap times are known	AUC ≈ 0.89. Measures pace, not just order
form	`drv_podium_rate_last10`	Only the 10 races *before* this one	AUC ≈ 0.87. Window chosen by the experiment in 6.2
form	`drv_avg_finish_last10`	Same	AUC ≈ 0.87

Group	Feature	Why it is valid at the cutoff	Evidence
form	`drv_dnf_rate_last10`	Same (DNFs of earlier races)	AUC ≈ 0.66. A reliability signal that Q3 shows varies a lot between teams
form	`team_podium_rate_last10`	Team's previous 10 races, computed at team-race level	AUC ≈ 0.88. Car strength regardless of driver
form	`team_best_finish_last10`	Same	AUC ≈ 0.89
standings	`drv_points_share`	Standings after the *previous* round	AUC ≈ 0.89. Comparable across points systems
standings	`drv_wins_before`	Same	AUC ≈ 0.75
standings	`team_points_share`	Same	AUC ≈ 0.89
experience	`drv_starts_before`	Count of earlier starts	AUC ≈ 0.69. Experienced drivers end up in top cars
experience	`drv_age`	Date of birth and race date	Weak on its own (≈ 0.58). Kept as a low-cost context feature
experience	`drv_circuit_podiums_before`	Podiums at this circuit in earlier seasons	AUC ≈ 0.70

Rejected candidates:

- `home_race`: Q2 found no grid-adjusted home effect and its AUC is 0.50.
- `pitlane_start`: about 1% of rows, no signal on its own (AUC 0.50). Already covered because `grid_pos` puts these cars at the back.
- `grid_vs_quali`: exactly `grid_pos − quali_pos`, so it is perfectly collinear with the two features we keep. Replaced by the `grid_penalty` flag.
- `drv_rank_before` and `team_rank_before`: correlation −0.95 / −0.92 with the points shares, and more missing values. They are near-duplicates.
- `drv_points_before` (raw points): the champion scored 95 points in 2009 and 575 in 2023, so raw points mean different things in different seasons.
- `field_size` and `race_round`: these are race-level constants. Every race has exactly three podiums, so they carry no signal by construction (AUC ≈ 0.5).
- `race_year`, `driverId`, `constructorId`, `circuitId`, car `number`: these are identifiers or a calendar trend. With a chronological split the model would have to extrapolate them to unseen seasons and new drivers. The form and standings features capture *how strong* a driver or team is, which is what actually transfers.

Strong correlations remain *between* groups (for example grid vs qualifying 0.97, team form vs team points share ≈ 0.9). This is expected, since they are different views of the same car's pace. It matters for the ablation, because removing one group may cost little when another group carries the same information. That is why the spec also suggests removing grid and qualifying together.

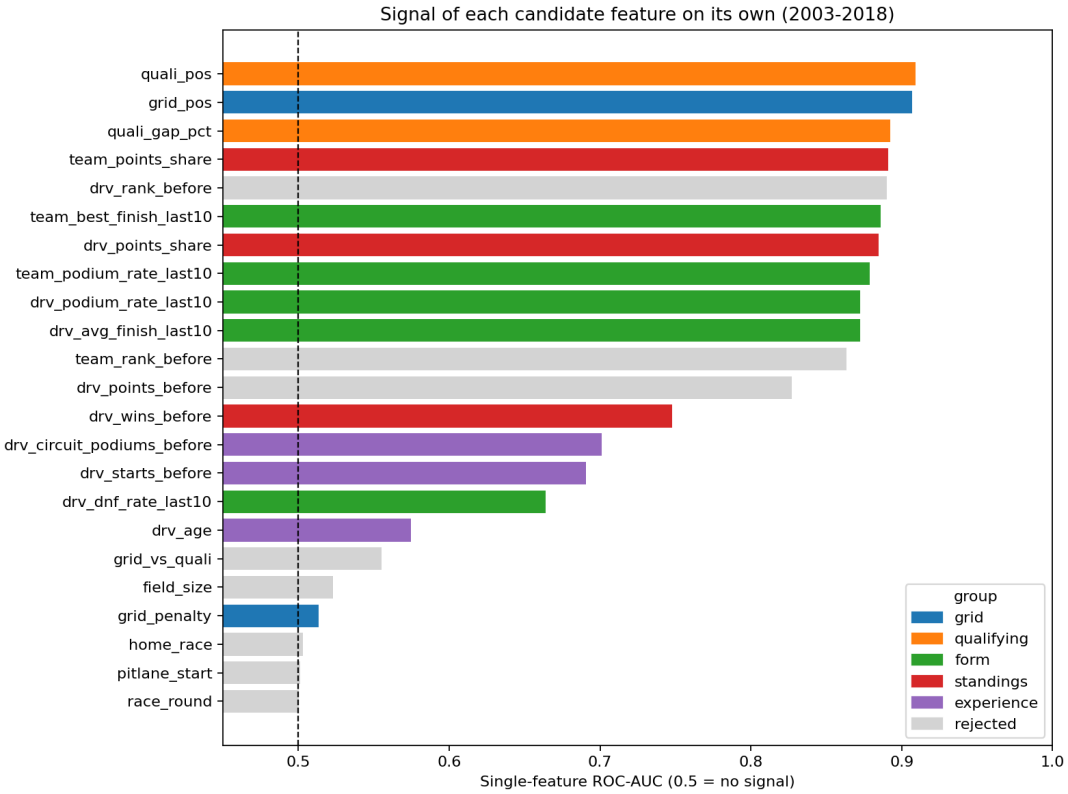


Figure 9. Univariate ROC-AUC of every candidate feature (2003–2018 only, so the validation/test seasons stay untouched).

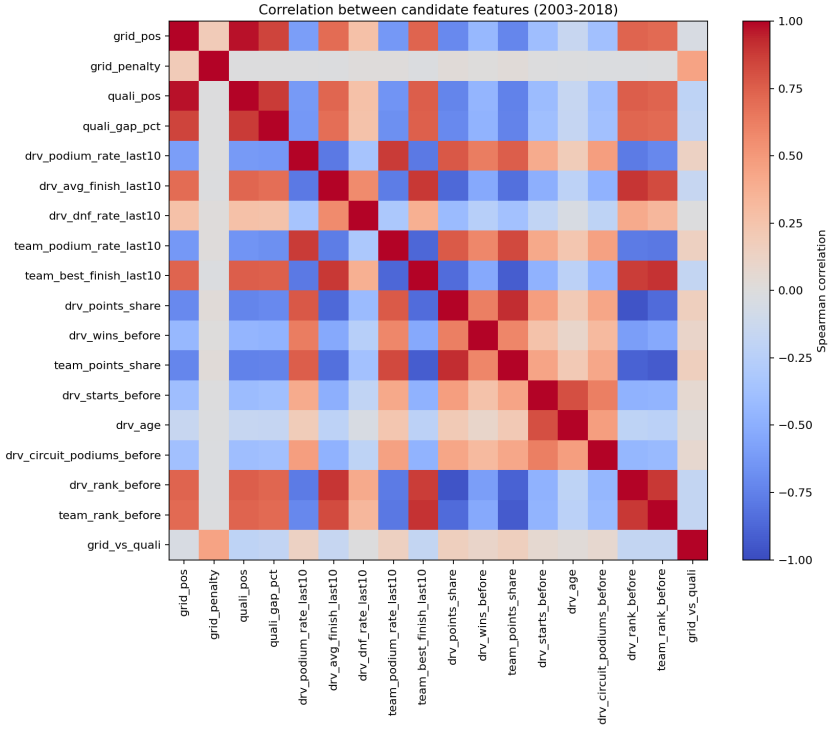


Figure 10. Correlation between features. The grid–qualifying block (0.97) is why the joint ablation and joint permutation importance are the informative numbers.

Leakage control and missing values

Three kinds of checks:

1. No post-race column is in the feature list.

2. For 300 random rows we recompute the driver form, team form and standings features **from scratch**, using only races with a smaller `race_seq`, and compare them with the vectorised values.
3. A driver's first race has no form, and season openers have no standings.

All three checks pass in the notebook: 300 random rows had their form, team form and standings features recomputed from scratch using only earlier races and matched the vectorised values; no post-race column is among the features; and no single pre-race feature separates podiums almost perfectly (AUC > 0.95).

The missing values are not random errors. Each one has a clear reason:

- **Points shares (~5%):** these are exactly the season-opener rows, because there are no standings before round 1.
- **Form (~1%):** the driver's or team's first ever race, so there is no history.
- `quali_gap_pct` **(~1.7%):** the driver has a qualifying position but no lap time (did not set a time, or the time was deleted).
- `quali_pos` / `grid_penalty` **(~0.3%):** there is no qualifying record.

We deliberately **do not impute here**. Imputation is a fitted preprocessing step, so it belongs in the modelling stage's pipeline and must be fitted on the training split only. Leaving the NaNs in also lets the modelling stage run the preprocessing-order experiments.

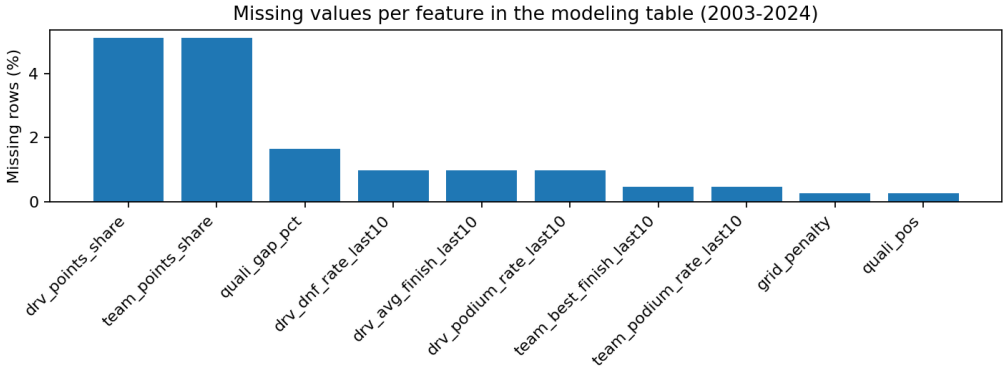


Figure 11. Where the modelling table has missing values — each has a structural reason.

5. Split and preprocessing

split	years	rows	races	drivers	podiums	podium_rate
train	2003-2018	6407	300	96	900	0.140
validation	2019-2021	1200	60	28	180	0.150
test	2022-2024	1359	68	28	204	0.150

Table 3. Chronological split. Rows within a season share cars and regulations, so a random split would leak; test seasons were not used for any feature, preprocessing or hyper-parameter decision.

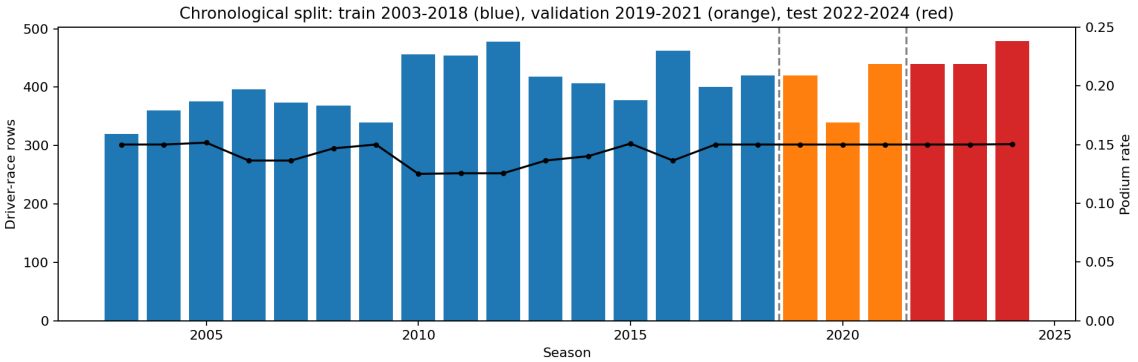


Figure 12. Split overview.

Preprocessing-order experiments

The brief asks for at least two preprocessing-order experiments. We run four real configurations plus two diagnostic ones, and we run them **before** choosing the final models, with two fixed cheap probes so the comparison is about preprocessing and nothing else:

- a **logistic regression** probe, which is sensitive to scaling, skew and imputation, and
- a **histogram gradient boosting** probe, which is invariant to any monotone transform of a feature, so for it only the imputation and the indicators can move the score.

Config	What changes
A	median impute → standardise (indicators on)
B	standardise (observed-only statistics) → impute in the scaled space
C	median impute → standardise, **without** missing indicators
D	`log1p` on the skewed counts → median impute → standardise
E (diagnostic)	config A but the scaler/imputer fitted on **train + validation** rows; this is a deliberately leaky diagnostic, and **never includes test**
F (diagnostic)	no preprocessing at all, raw NaNs fed to the tree model's native NaN handling

E is not a candidate. It is the leakage control: it illustrates the effect of using validation information in preprocessing; it never consults the test block, and is excluded from model/preprocessor selection.

config	probe	leakage_safe	val_pr_auc	val_roc_auc	val_f1
A_impute_then_scale	logreg	True	0.7216	0.9375	0.7246
A_impute_then_scale	hist_gb	True	0.6741	0.9284	0.7068
B_scale_then_impute	logreg	True	0.7218	0.9376	0.7264
B_scale_then_impute	hist_gb	True	0.6741	0.9284	0.7068
C_no_missing_indicators	logreg	True	0.7151	0.9347	0.7212
C_no_missing_indicators	hist_gb	True	0.6728	0.9282	0.7113
D_log_then_impute_scale	logreg	True	0.7088	0.9364	0.7246
D_log_then_impute_scale	hist_gb	True	0.6741	0.9284	0.7068
E_LEAKY_train_plus_val	logreg	False	0.7216	0.9375	0.7264
E_LEAKY_train_plus_val	hist_gb	False	0.6909	0.9304	0.7026

config	probe	leakage_safe	val_pr_auc	val_roc_auc	val_f1
F_raw_native_nan	hist_gb	True	0.6804	0.9281	0.7016

Table 4. Preprocessing-order experiments (validation). Configuration E is an illustrative train+validation leakage control; it does not use test data.

Selection: configurations within 0.005 mean validation PR-AUC of the best are treated as tied (the same probe moves by about that much when library versions change), and ties are broken in favour of keeping missing-value indicators, because missingness is informative (season openers have no standings). The selected configuration is B (scale on observed values, then impute, with indicators).

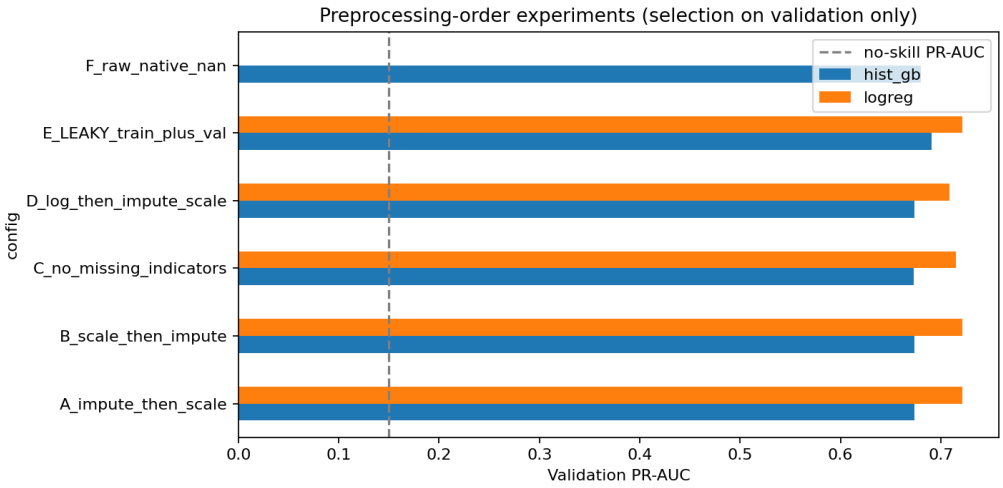


Figure 13. Preprocessing-order experiments.

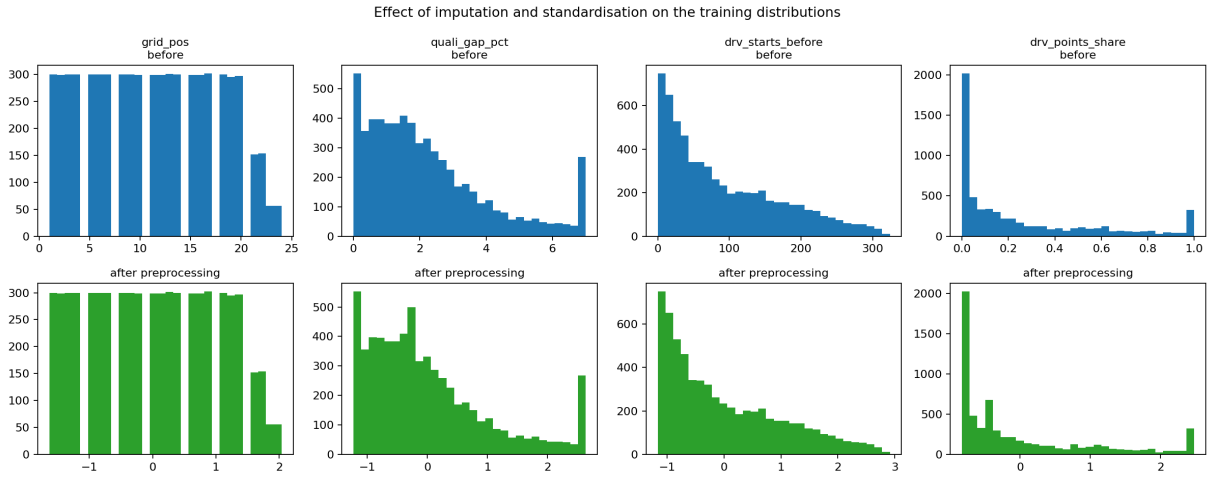


Figure 14. Effect of the preprocessing on the distribution and range of the data.

6. Modelling

Four model families were compared with a grid-position baseline: logistic regression, random forest, histogram gradient boosting and a shallow neural network. Each family is tuned on validation PR-AUC only; the decision threshold is tuned on validation as well.

model	split	pr_auc	roc_auc	f1	precision	recall	top3_hit_rate	threshold
baseline_grid_position	validation	0.676	0.917	0.717	0.717	0.717	0.717	0.870
baseline_grid_position	test	0.647	0.914	0.649	0.650	0.647	0.652	0.870
hist_gradient_boosting	validation	0.737	0.938	0.734	0.718	0.750	0.722	0.425
hist_gradient_boosting	test	0.638	0.925	0.641	0.661	0.623	0.623	0.425
logistic_regression	validation	0.729	0.938	0.727	0.683	0.778	0.678	0.405
logistic_regression	test	0.698	0.934	0.662	0.620	0.711	0.662	0.405
random_forest	validation	0.746	0.941	0.734	0.724	0.744	0.711	0.430
random_forest	test	0.695	0.936	0.655	0.679	0.632	0.647	0.430
shallow_nn	validation	0.717	0.936	0.731	0.662	0.817	0.706	0.320
shallow_nn	test	0.679	0.926	0.656	0.571	0.770	0.618	0.320

Table 5. Model comparison on validation and test (PR-AUC primary).

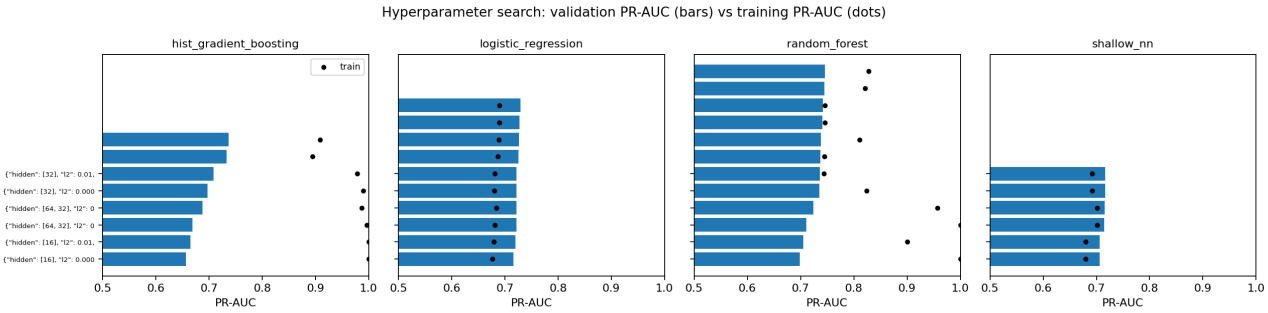


Figure 15. Hyper-parameter search: validation PR-AUC (bars) vs training PR-AUC (dots) — the gap shows overfitting.

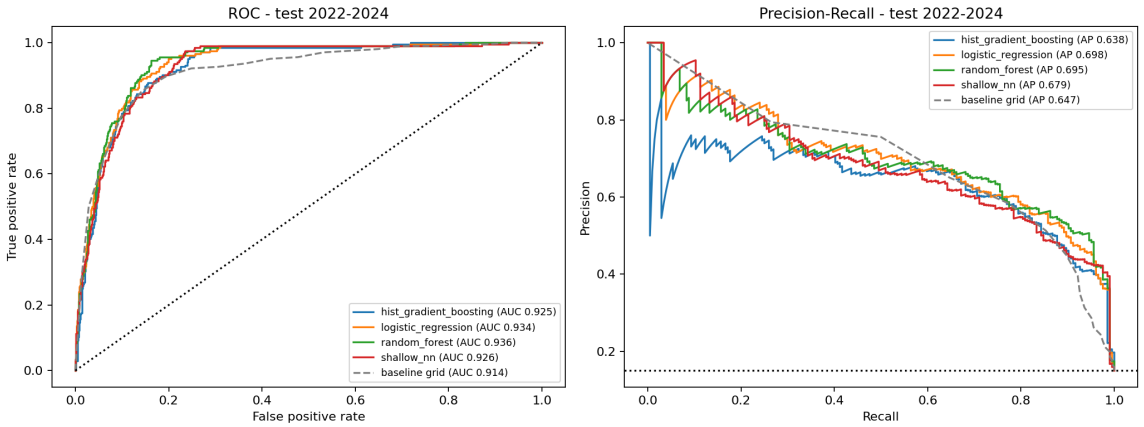


Figure 16. ROC and precision–recall curves on the test block.

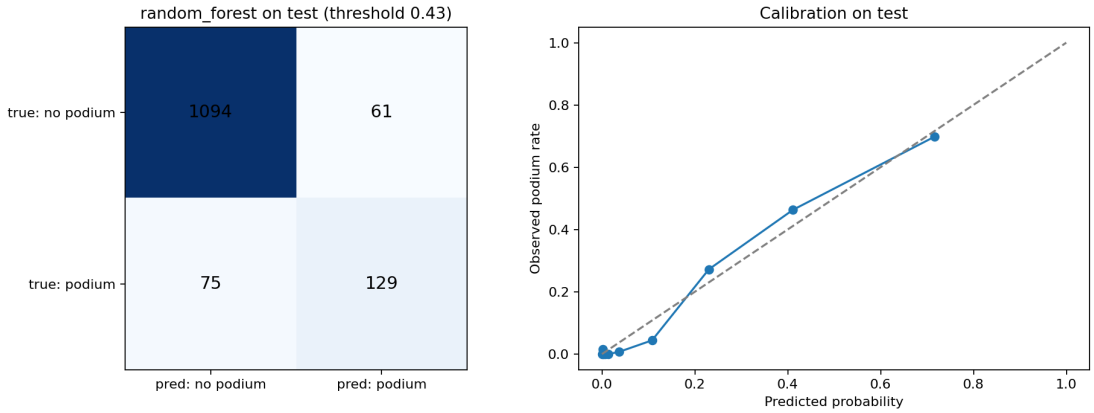


Figure 17. Confusion matrix and calibration of the selected model.

7. Experiments

Feature-group ablation

An ablation removes one group of features at a time and retrains, so we can measure what each group actually contributes *after* training (the feature-engineering stage's evidence justified the features *before* training; these two things are not the same and both are required).

The columns removed for each group come straight from `spec["ablation_drop"]`, so the definitions match the feature-engineering stage's. Two details to defend:

- `grid_penalty` is dropped in the **qualifying** ablation as well, because it is derived from qualifying position.
- `grid_and_qualifying` removes both groups together. The feature-engineering stage measured a 0.97 correlation between them, so removing either on its own mostly measures redundancy, not importance. This combined run is the informative one.

Everything is retrained from scratch: a fresh `Preprocessor` fitted on the reduced training columns (so the imputation statistics match the reduced feature set) and the selected model family with its selected hyperparameters.

ablation	n_features	val_pr_auc	test_pr_auc	test_roc_auc	test_f1	delta_test_pr_auc
none (full feature set)	15	0.7457	0.6951	0.9362	0.6548	0.0000
drop grid	13	0.7253	0.6816	0.9332	0.6508	-0.0135
drop qualifying	12	0.7282	0.6939	0.9351	0.6482	-0.0013
drop form	10	0.7424	0.7000	0.9382	0.6835	0.0049
drop standings	12	0.7433	0.6788	0.9355	0.6886	-0.0163
drop experience	12	0.7505	0.6896	0.9350	0.6730	-0.0055
drop grid_and_qualifying	11	0.6490	0.5774	0.8943	0.5565	-0.1177

Table 6. Retraining without one feature group at a time (selected model family and hyper-parameters; fresh preprocessor on the reduced columns).

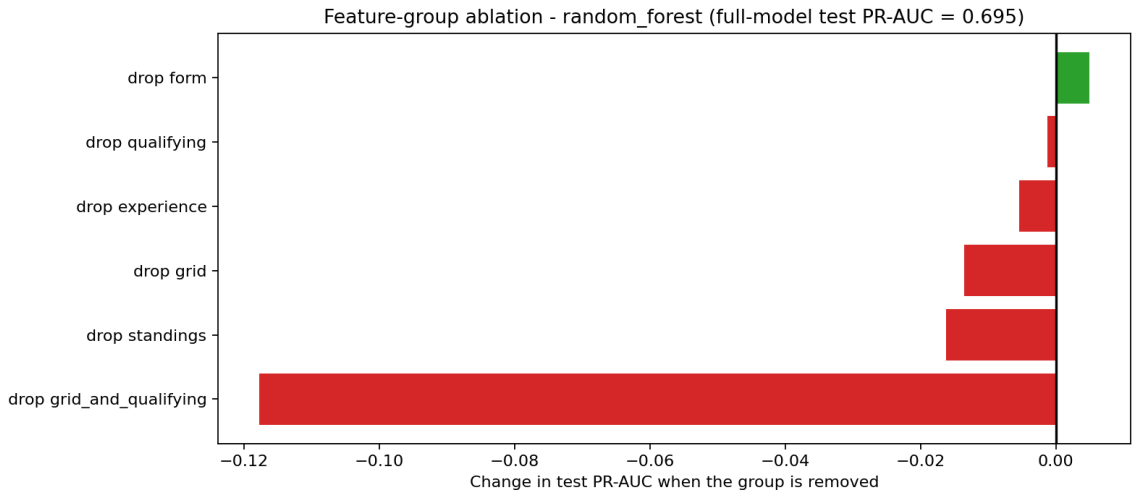


Figure 18. Ablation results.

Era sensitivity and refit check

season	rows	roc_auc	pr_auc	f1	top3_hit_rate
2022.0	440.0	0.943	0.664	0.672	0.667
2023.0	440.0	0.928	0.759	0.626	0.636
2024.0	479.0	0.932	0.648	0.662	0.639

Table 7. Test performance by season: the three test seasons differ, which is the cost of the 2022 regulation change.

fit_on	roc_auc	pr_auc	f1	top3_hit_rate
train 2003-2018 (reported model)	0.936	0.695	0.655	0.647
train + validation 2003-2021	0.936	0.680	0.650	0.652

Table 8. Refitting on train + validation does not improve the test score, so the reported model stays the one trained on 2003–2018.

8. Explainable AI

The selected model (random_forest) is explained on the processed design matrix (standardised features plus missing indicators, in the training column order) using the test block (2022–2024). Permutation importance measures the drop in PR-AUC when columns are shuffled (20 repeats); SHAP uses TreeExplainer on the selected random forest (additivity verified numerically); LIME and SHAP waterfalls explain individual predictions.

Global: permutation importance

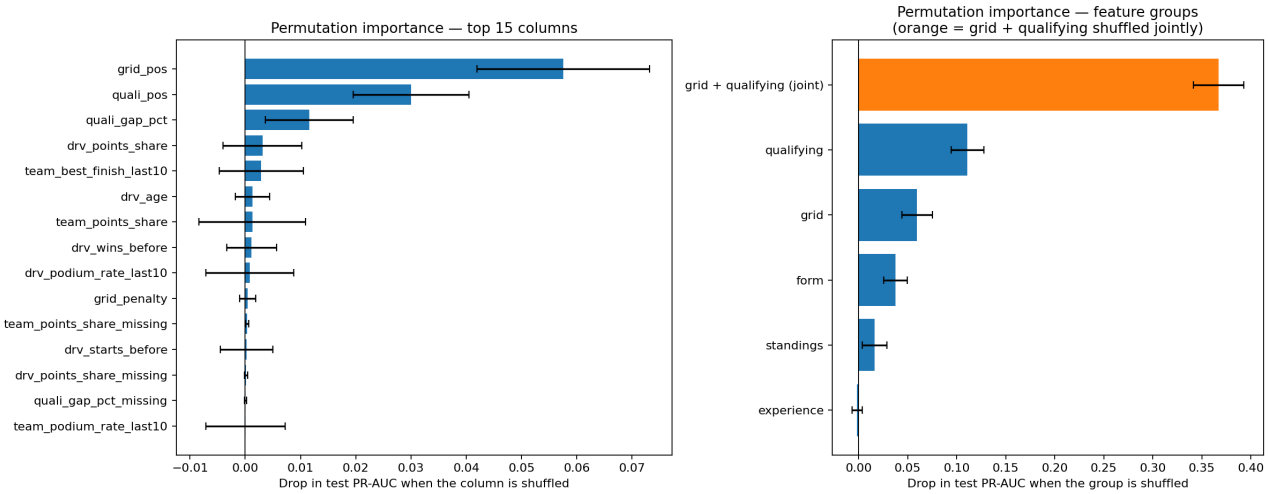


Figure 19. Permutation importance of single columns (left) and feature groups (right).

Reading the result. Shuffling `grid_pos` costs about 0.06 PR-AUC (roughly 8% of the model's 0.70), `quali_pos` about 0.03 and `quali_gap_pct` about 0.01. Every other *single* column costs less than its own shuffle-to-shuffle standard deviation, i.e. nothing measurable. That does **not** mean the other features are useless — they are redundant with each other (the form features and the points shares describe the same car). The group view shows it: form and standings cost about 0.04 and 0.02 when shuffled together, and shuffling grid **and** qualifying jointly costs about 0.37 — six times the grid group alone and more than twice the sum of the two single groups. The two groups carry the same information, so each one covers for the other when it is shuffled alone. Experience features (starts, age, circuit podiums) cost nothing at all.

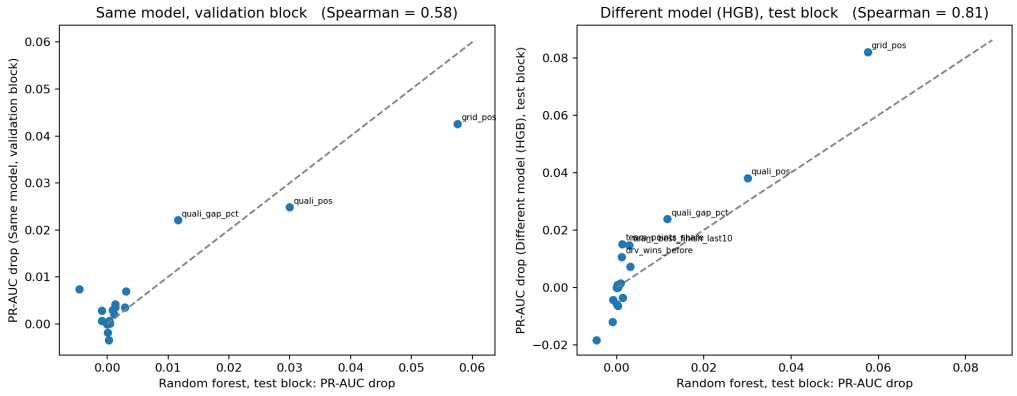


Figure 20. Stability of the ranking: same model on validation (left) and a different model family (right).

Reading the result. The top three columns (`grid_pos`, `quali_pos`, `quali_gap_pct`) are identical on the validation block and for a different model family. Beyond the top three the rank correlation is only moderate because the tail is noise around zero, so we interpret the top of the ranking and the group-level result, **not** the order of the tail.

Global: SHAP

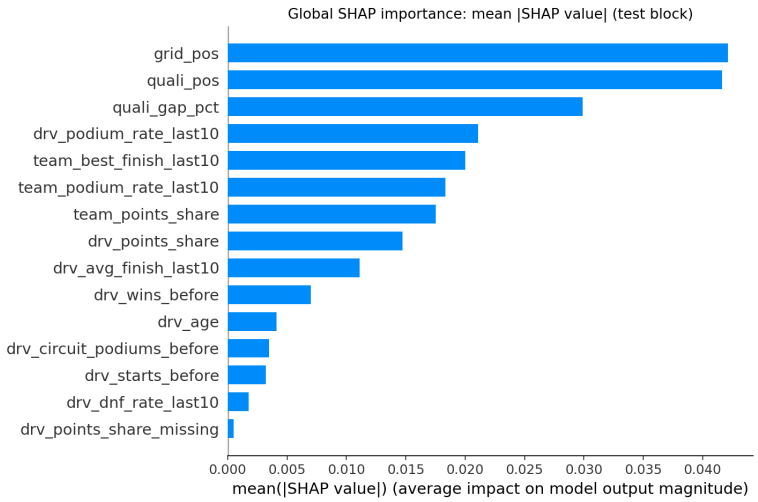


Figure 21. Mean |SHAP| per feature.

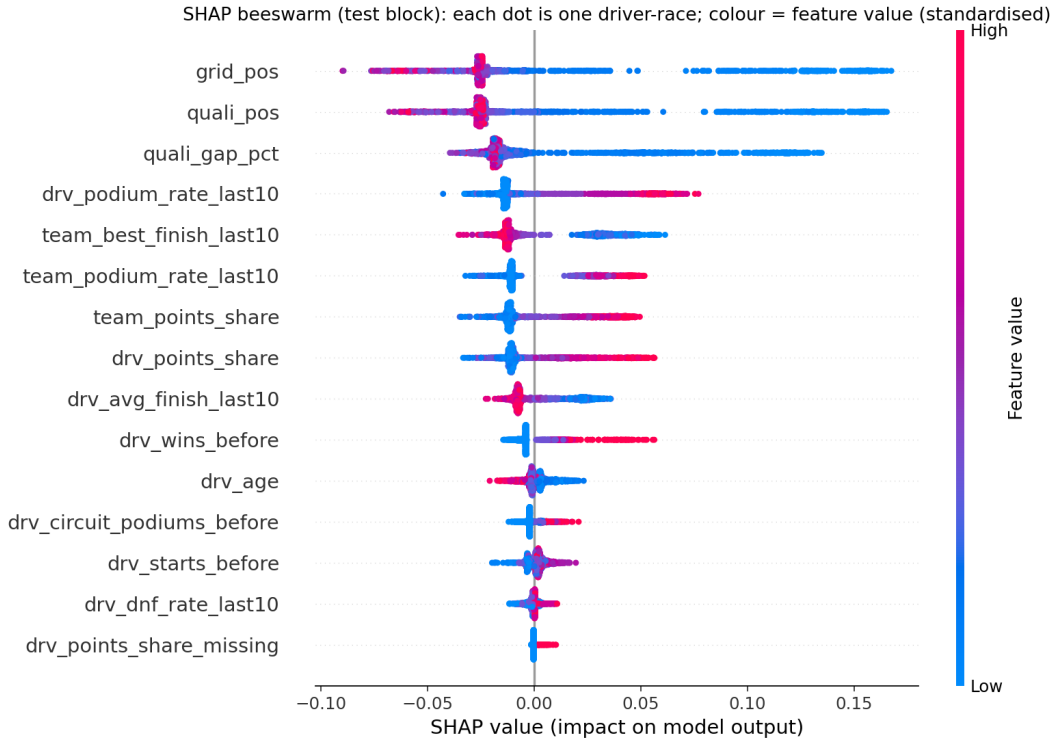


Figure 22. SHAP beeswarm.

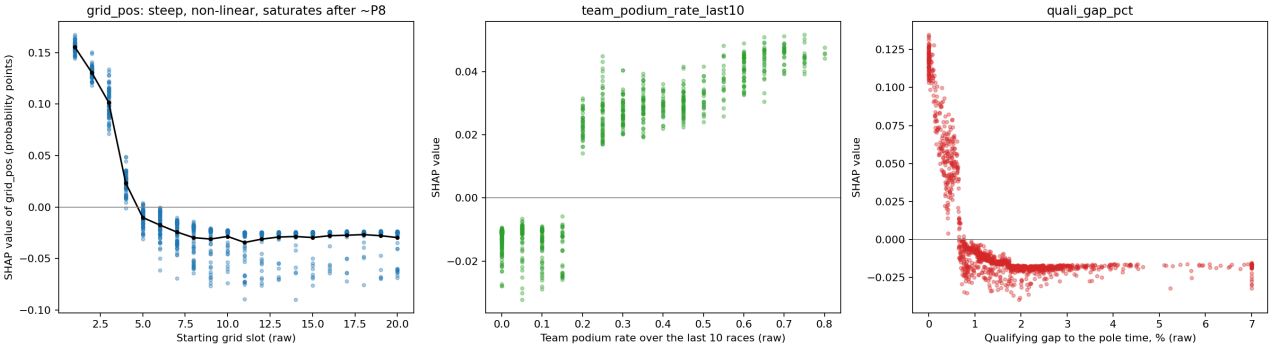


Figure 23. SHAP value against the raw feature value (black line: mean per grid slot).

Reading the result. The three qualifying/grid columns carry roughly half of all SHAP mass; form and standings columns share most of the rest, and experience columns are small. The shapes matter more than the ranks:

- `grid_pos` is **steep and saturating**: starting P1–P3 adds roughly +0.15 to +0.10 probability points, P4 is about neutral, and from P5 back the contribution is flat at about −0.03. The model learned "front of the grid matters a lot, the exact slot in the pack does not" — consistent with Part 4, where the podium rate fell from ≈ 79% on pole to ≈ 5% by P8. A single linear coefficient could not express this.
- `team_podium_rate_last10` shows a **jump** rather than a slope: below about 0.2 the car is penalised, above it the contribution rises gently. `quali_gap_pct` is large only for gaps below about 0.7% of the pole time and flat beyond that.
- Group-level SHAP agrees with the permutation result: the grid + qualifying pair is by far the largest contributor.

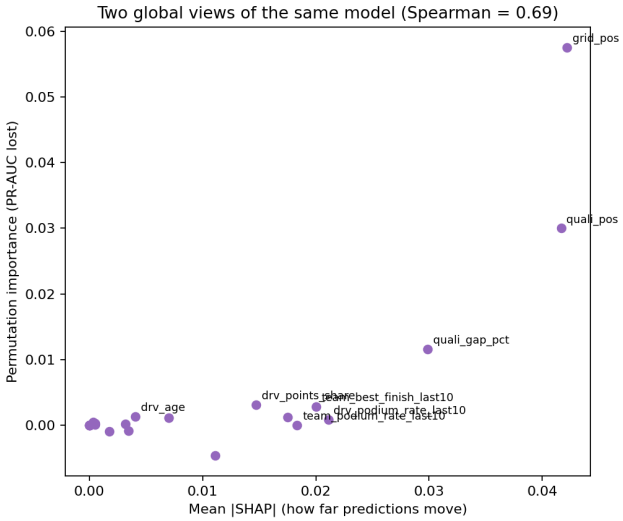


Figure 24. Permutation importance vs mean |SHAP|.

Reading the result. The two global methods agree on the top three columns. They disagree on the middle of the ranking in an instructive way: the form and standings columns (`drv_podium_rate_last10`, `team_best_finish_last10`, `team_podium_rate_last10`, the points shares) have sizeable SHAP values — the model **uses** them to move predictions — but shuffling any one of them costs no measurable held-out PR-AUC. SHAP says what the model leans on; permutation importance says what that reliance is worth on unseen data. For these redundant columns the answer is "little, individually", which matches the Part 7 ablation where dropping the whole form group does not hurt test PR-AUC.

Local explanations

Three cases were chosen **by outcome** (not by how well they explain): the most confident correct podium call, the most confident wrong podium call and the biggest surprise.

case	row	driver	race	grid	predicted_prob	predicted_podium	actual_podium
A_confident_correct	287	Max Verstappen	Dutch Grand Prix 2022	1	0.907	1	1
B_confident_wrong	1086	Max Verstappen	Austrian Grand Prix 2024	1	0.893	1	0
C_biggest_surprise	690	Pierre Gasly	Dutch Grand Prix 2023	12	0.001	0	1

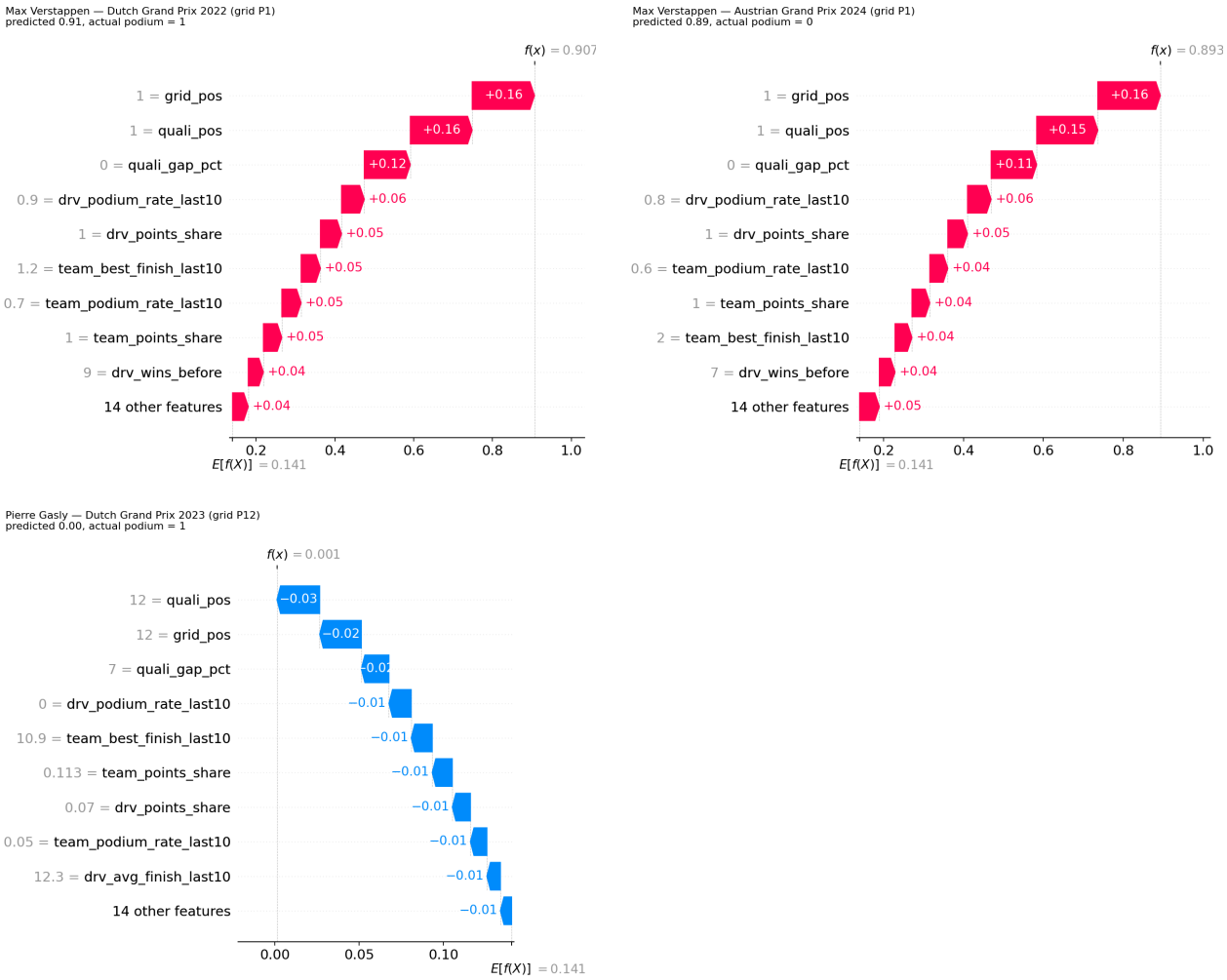


Figure 25. SHAP waterfalls for the three cases (raw feature values on the left, contributions in probability points).

Reading the waterfalls. *Case A* (Verstappen, Dutch GP 2022): every one of the top features pushes toward a podium, led by the front-row start; nothing pulls back — a textbook easy call. *Case B* (Verstappen, Austrian GP 2024) has almost the same profile, so the model is almost equally sure, and it was **wrong** (no podium). The explanation shows why no feature-level fix exists: all the pre-race information genuinely said "podium", and what happened on race day is not in the features. This is a limit of the problem (pre-race information only), not a bug in the explainer. *Case C* (Gasly, Dutch GP 2023) was the opposite: starting 12th with a team form of 0.05, every feature pushes down, the model gave him ≈ 0.1% and he finished on the podium — an upset the features cannot anticipate.

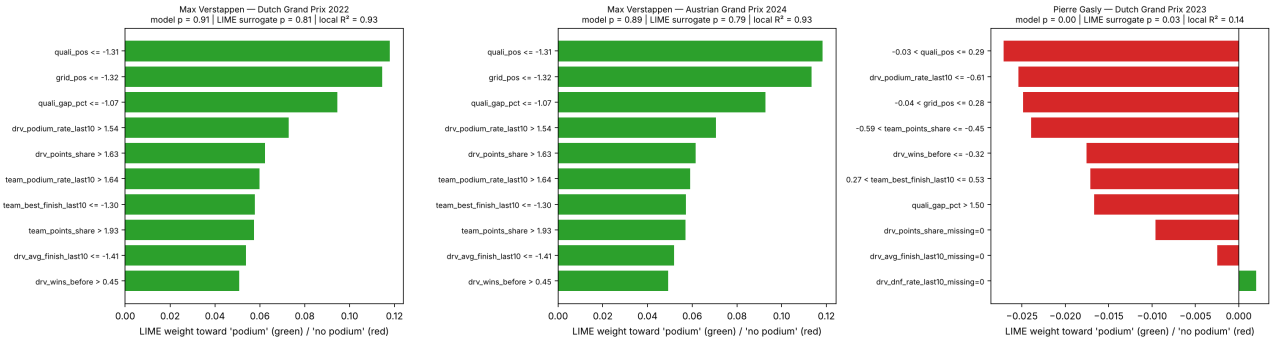


Figure 26. LIME explanations of the same three cases.

Reading the LIME result. In the fidelity study the setting with decile bins and a narrower kernel reproduced the real model at the instance best (mean error of the surrogate probability ≈ 0.03), so it is used for the three cases. For the two confident cases the surrogate is faithful (local $R^2 \approx 0.93$,

surrogate probability close to the model's) and it highlights the same features as SHAP. For the surprise case the local R^2 is low (≈ 0.14): around a probability of ≈ 0 the model is nearly flat, so a linear surrogate has little to fit and its weights are small — LIME's output for that case should be treated as weak evidence.

case	shared_top5	sign_agreement_on_shared	lime_seed_stability_top5
A_confident_correct	5	5/5	0.86
B_confident_wrong	5	5/5	0.92
C_biggest_surprise	3	3/3	0.8

Table 9. Agreement between SHAP and LIME and LIME's stability across five seeds.

Reading the agreement table. For the confident cases LIME and SHAP share all five of their top-5 features with the same signs; they differ mainly in the *order* of the first two (grid vs qualifying position — near-duplicates). For the surprise case they share three of their five features, with matching signs, and differ on the minor ones, where LIME picks different weak contributors. LIME is also seed-dependent: across five seeds the top-5 sets overlap by 0.8–0.9, not 1.0. We therefore trust the features on which SHAP and LIME agree, and treat LIME-only features as unconfirmed.

Feature importance is not causality

Everything in this part describes **what the model uses**, not **what makes a driver finish on the podium**. Three pieces of evidence from this notebook show why the distinction matters here, not just in principle.

1. **Common cause.** Grid position tops the importance rankings, but starting P1 does not *cause* a podium in the way an intervention would. The grid is itself an outcome of car pace and driver skill on Saturday, and those same factors also drive Sunday's result. Moving a midfield driver to P1 by decree would not give him the car that earned the slot.
2. **Home advantage (Part 3, Q2).** The raw podium rate looks like it says something about home races, and the effect disappears (Mantel–Haenszel odds ratio ≈ 0.97, interval containing 1) as soon as grid position is controlled for. A model fed a `home_race` flag would have assigned it importance for exactly the wrong reason, which is why it is not a feature.
3. **Correlated features split credit arbitrarily.** The table below shows that shuffling grid *or* qualifying alone costs little, because the other substitutes, while shuffling both together costs far more. Reading the single-column numbers as "qualifying doesn't matter" would be a mistake; the ablation in Part 7 shows the same thing from the retraining side.

9. Inference

The final prediction function. It takes one record (a dict) or several (a DataFrame) containing any subset of the 15 pre-race features, applies the **fitted** preprocessor (training-set statistics only, missing values imputed and flagged exactly as in training), scores with the selected model and returns the podium probability, the decision at the validation-tuned threshold and a list of which inputs had to be imputed. Unknown extra columns are ignored and reported.

input	P(podium)	podium?	# imputed	imputed fields	top SHAP feature
complete record	0.397	0	0	none	grid_pos
partially-missing record	0.103	0	5	grid_penalty, quali_pos, quali_gap_pct, drv_points_share, team_points_share	grid_pos

Table 10. Demonstration of `predict_podium`*: a fully observed 2023 front-row record (0 missing inputs) and the same record with qualifying data, penalty flag and points shares removed. The function imputes with training-set statistics, reports which fields were imputed and keeps working; the probability drops as information is removed.*

10. Limitations

- **Era shift.** The test block (2022–2024) is the ground-effect era; no training season is like it. The per-season test breakdown in Part 7 quantifies the cost, and explanations computed on the test block describe the model's behaviour *in that era*.
- **Historical qualifying coverage.** Complete qualifying data exists only from 2003, which sets the start of the modelling table; 1994–2002 is partial and earlier seasons have none (Part 1, Part 4). Qualifying features cannot be used for older seasons.
- **Correlated groups.** Grid and qualifying correlate at 0.97, so single-feature importance splits credit between them arbitrarily; use the joint rows.
- **Validation reuse.** Validation chose the preprocessing, hyperparameters, early stopping and threshold, so its scores are mildly optimistic; test scores are not.
- **Independent scoring.** Each driver is scored on his own; nothing stops the model predicting five podiums in one race. A per-race ranking model is the natural next step.
- **Imputation of season openers** is a modelling convention (median plus a missing indicator), not a fact about those races.
- **Explanations are not causes.** See the dedicated section in Part 8.

11. Reproducibility

The single notebook `f1_podium_prediction_final.ipynb` is designed to Run All from the raw CSVs (runtime depends on the available CPU and LIME installation), with every random step seeded (SEED = 42). On Kaggle, attach the dataset and press Run All; `make_report.py` rebuilds this PDF from the files the notebook writes so the report always matches the run.